

Python I: values, functions, and the loop

The first of six sessions on the tools, and the one that assumes least. By the end of today you will be able to write a function that takes a list of numbers and returns their average — which is a smaller thing than it sounds, and the exact thing everything else in this course is built on.

Unit 1 • Foundations

80 minutes

Lab 1 • five small functions

Laptops open

[Who this is for](#) [The notebook](#) [Values and names](#) [Functions](#) [Choosing](#) [The loop](#) [All four at once](#) [Lab 1](#) [Before Tuesday](#)

OPENING • 6 MINUTES

Two kinds of student are in this room

0

Some of you have written a few hundred lines of Java or C. Some of you have never written a line of anything. The diagnostic told me roughly how many of each, and this session is built for the second group — deliberately, and without apology to the first.



If you have programmed before

The first twenty minutes will be slow. Use them: the syntax is different from Java in ways that will bite you later, and the last part of this session — the loop with an accumulator — is the exact shape of the training loop you will write in session 16. Read the extension problems at the end of Lab 1 rather than sitting idle.



If you have not

You are not behind. Unit 1 is six sessions — nearly a fifth of the course — and it exists because arriving without programming is normal here. Two things will help more than anything else I can say. Type every line yourself rather than copying it, because your hands learn what your eyes do not. And when something breaks, read the last line of the error message before you do anything else; it is usually a plain English sentence.

TODAY'S TARGET, STATED EXACTLY

By the end of this session you can write this, and explain every line of it:

PYTHON

```
def average(values):  
    total = 0  
    for v in values:  
        total = total + v  
    return total / len(values)
```

Five lines. A value, a name, a function, a loop, and a return. Everything in unit 2 is this shape with more arithmetic in the middle.

PART 1 • 5 MINUTES

The notebook, and why we use one

1

Every lab in this course is a Jupyter notebook. Two minutes on the mechanics, and one warning that will save you an afternoon later.

Anaconda Prompt (Windows) • Terminal (macOS / Linux)

SHELL

```
conda activate csc4315  
jupyter lab
```

A notebook is a list of cells. Shift+Enter runs a cell and moves on. Variables persist between cells, which is the whole point: you can compute something expensive once and then explore it for twenty minutes without recomputing.



The warning: cells remember, in the order you ran them

A notebook's state depends on the order you *ran* the cells, not the order they appear on screen. You can delete a cell and still have its variable. You can have working code that fails completely for the person you send it to.

Before you submit any lab: Kernel → Restart Kernel and Run All Cells.

If it does not run clean from top to bottom, it is not finished. I will run your notebooks this way, and so will every employer you ever have.

PART 2 • 10 MINUTES

Values, and the names we give them

2

Four kinds of value, one operator that surprises people, and one rule about names.

A **value** is a piece of data. Python has many kinds; this course uses four, and you will meet three of them in the next ten minutes.

PYTHON

```
temperature = 36.6      # a float – a number with a decimal point
count        = 42       # an int – a whole number
name         = "Aya"    # a str – text, in quotes
is_ready     = True     # a bool – True or False, capitalised
```

The `=` sign is not equality. It means *give this value the name on the left*. Read it as "becomes", never as "equals" — the difference will matter in ten minutes when you write `total = total + v`, which is nonsense as equality and perfectly sensible as an instruction.

The name is a label, not a box

A name does not contain a value; it points at one. Today that distinction costs you nothing. In session 5, when you meet lists, it becomes the single most common cause of bugs that make no sense — so meet it now while it is harmless.

PYTHON

```
a = 5
b = a      # b now points at the same 5
a = 7      # a now points at a 7 instead
print(a, b) # 7 5 – b did not follow a
```

The operator that catches everyone

PYTHON

```
7 / 2      # 3.5   true division – always gives a float
7 // 2     # 3     floor division – throws the remainder away
7 % 2      # 1     the remainder itself
```

Where this will bite you

Use `//` when you mean "how many whole mini-batches fit in this dataset" — session 16. Use `/` everywhere else. Silently getting 3 where you meant 3.5 is a classic way to make a loss function quietly wrong, and quietly wrong is much worse than broken, because nothing tells you.

PART 3 • 14 MINUTES

A function is a machine you name once and use often

3

This is the most important part of the session. Everything you write for the rest of the term is a handful of functions.

A function takes something in, does work, and hands something back. Three words carry it: `def` to define, the names in the brackets to receive, and `return` to hand back.

PYTHON

```
def double(x):          # def · the name · what it receives
    return x * 2        # what it hands back

print(double(5))        # 10
print(double(1.5))      # 3.0 — the same machine, a different input
```



The indentation is the syntax

In Java or C, braces mark what belongs to the function. In Python, the indentation *is* the structure — four spaces, and every line at that depth is inside the function. This is not a style preference; get it wrong and the program means something else. Your editor will indent for you after a line ending in a colon.

Two arguments, and a name that means nothing to Python

PYTHON

```
def rectangle_area(width, height):
    return width * height

print(rectangle_area(3, 4))    # 12
```

Python does not know what "width" means. The names are for the person reading, and the person reading in November will be you.

`rectangle_area(w, h)` runs identically and tells you nothing.

The difference between printing and returning

This trips up almost everyone in their first week, so meet it deliberately.

PYTHON

```
def show_double(x):
    print(x * 2)        # shows it on screen, hands back nothing

def give_double(x):
    return x * 2        # hands it back, shows nothing

result = show_double(5) # prints 10 · result is None
result = give_double(5) # prints nothing · result is 10
print(give_double(5) + 1) # 11 — you can only do this with a return
```



Print is for you. Return is for the program.

A function that prints its answer instead of returning it is a dead end: nothing else can use the result. Every function you write in this course returns. If you want to see a value as well, print the result of calling the function — not from inside it.

Choosing: if, else, and comparison

PYTHON

```
def classify(score):
    if score >= 50:
        return "pass"
    else:
        return "fail"

print(classify(72), classify(31))  # pass fail
```

The comparisons are `>`, `<`, `>=`, `<=`, `==` for equal and `!=` for not equal. Note the double equals: one `=` gives a name to a value, two `==` asks a question. Python will tell you if you confuse them, which is more than C will do.

Return exits immediately

PYTHON

```
def classify(score):
    if score >= 50:
        return "pass"
    return "fail"  # only reached if the if did not fire
```

Both versions are correct. The second is common in real code because `return` ends the function on the spot — nothing after it runs. You saw exactly this shape in the diagnostic, in the `even / odd` question.



Do this now — two minutes, on paper

Write a function `bigger(a, b)` that returns the larger of its two arguments. Do not run it. When you think it is right, trade papers with the person beside you and find one thing wrong with theirs — a missing colon, a missing return, an indentation that is off. Finding other people's mistakes is how you learn to see your own.

The loop, and the accumulator inside it

The single most important fifteen minutes of unit 1. Everything from session 9 onward is a loop with an accumulator.

A **list** is several values under one name. You will meet lists properly on Tuesday; today you only need to be able to walk along one.

PYTHON

```
scores = [70, 85, 60, 95]

for s in scores:
    print(s)  # 70 · 85 · 60 · 95, one per line
```

Read `for s in scores` as: *take each value in scores, call it s, and run the indented lines with it*. The name `s` is yours to choose and exists only inside the loop.

The accumulator: a name that survives the loop

Printing each value is not useful. What you almost always want is to *build up* an answer as you go. That needs a name created *before* the loop, which the loop then updates.

PYTHON

```
scores = [70, 85, 60, 95]

total = 0          # before the loop - this is the accumulator
for s in scores:
    total = total + s  # inside - updated once per value
print(total)        # 310 - after the loop, the answer is here
```



Trace it by hand before you run it

Draw two columns, `s` and `total`, and fill in a row per pass: (70, 70), (85, 155), (60, 215), (95, 310). Do this on paper for every loop you do not immediately believe. It takes ninety seconds and it is the skill the whole of session 14 depends on — the diagnostic's first question was exactly this and it is worth being certain about.

Three details decide whether an accumulator works, and each is a whole class of bug:

1

Start it before the loop.

Put `total = 0` inside and it resets on every pass, leaving you with the last value instead of the sum.

2

Start it at the right value.

Zero is right for a sum. For "the largest so far", zero is wrong the moment the data contains negatives — start from the first element instead. This was bug two in the diagnostic.

3

Return after the loop, not inside it.

A `return` indented into the loop body ends the function on the first pass. This was bug three in the diagnostic, and it is the one that separates people who trace code from people who read it.

Counting instead of summing

PYTHON

```
def count_passes(scores):
    n = 0
    for s in scores:
        if s >= 50:
            n = n + 1      # only when the condition holds
    return n

print(count_passes([70, 40, 85, 30]))  # 2
```

Same skeleton, different accumulator: a counter rather than a sum, and an `if` deciding whether to update it. Sum, count, largest, smallest — all four are this shape, and between them they cover most of what you will write before session 9.



A shortcut you will see everywhere

`total += s` means exactly `total = total + s`, and `n += 1` means `n = n + 1`. Use whichever you find clearer; the long form is easier to trace when you are learning, so this course writes it out and you should too, for now.

PART 6 • 5 MINUTES

6

All four ideas, in five lines

Back to the target from the opening. You now have every piece of it.

PYTHON

```
def average(values):      # a function, receiving a list
    total = 0              # an accumulator, before the loop
    for v in values:       # the loop
        total = total + v  # the update
    return total / len(values) # hand the answer back
```

`len(values)` gives the number of items — one of a handful of built-in functions you can simply use. And there is a bug in this version, which you should be able to find by asking one question: what happens if `values` is empty?

An empty list makes `len(values)` zero, and dividing by zero crashes with `ZeroDivisionError`. The fix is one `if` before the division:

PYTHON

```
def average(values):
    if len(values) == 0:
        return 0
    total = 0
    for v in values:
        total = total + v
    return total / len(values)
```

Thinking about the empty case, the zero case and the negative case before running anything is a habit worth building now. In session 20 it becomes the difference between a pipeline that works and one that silently drops rows.

WHY THIS SESSION MATTERED

In session 8 you will write a layer of a neural network. It is a function that receives numbers, loops over them accumulating a weighted sum, and returns the result. In session 16 you will write a training loop: an accumulator holding the loss, updated once per batch, returned at the end. You have just learned the shape of both. Everything after this is more arithmetic inside the same five lines.

Five functions, four lines or fewer

Lab 1 is a separate handout — open `CSC4315_Lab01_Five_Functions.html` or the printed copy. Five short functions, in order of difficulty, every one of them a variation on what you have seen in the last hour.



The rule for every lab in this course

You finish it in the room.

If you are stuck for more than three minutes, raise your hand — that is what the time is for, and asking early is the efficient move rather than the embarrassing one. Nothing in Lab 1 is a prerequisite for Tuesday: session 5 starts from a state the whole room shares, whether or not your lab ran.

Before Tuesday

1. **Finish Lab 1** if you did not get through all five in the room. Nobody is checking, and you will feel the difference on Tuesday if you skip it.
2. **Re-type the `average` function from memory**, without looking. Get it wrong, look, and do it again. Ten minutes, and it is worth more than an hour of reading.
3. **Optional, for anyone who found today easy:** write `largest(values)` without using `max`, and make it work when every number is negative. Bring it if you would like me to look at it.



Session 5 — Tuesday 22 September, 16:00

Python II: lists, slicing, comprehensions. Lists properly this time — indexing, the half-open slice that trips up everyone including experienced programmers, nested lists as grids, and the comprehension as a shorter loop. The grid we build on Tuesday is the same object as an image in session 23, which is not a coincidence.